TrackExtrapolation Gen_1 V1
Exhaustive Data Generation and Training Protocol

Repository Protocol Document

April 1, 2026

Contents

1 Scope and Purpose **2**
- 1.1 What is covered . . . 2
- 1.2 What is not currently active in V1 training . . . 2

2 Code and File Inventory (Source of Truth) **2**

3 Physics Model Used for Data Generation **2**
- 3.1 State Definition . . . 2
- 3.2 Lorentz-force equations in z-parameterization . . . 3
- 3.3 Magnetic field source and interpolation . . . 3
- 3.4 Integrator settings . . . 3
- 3.5 Track-level validity checks . . . 3

4 Sampling Protocol for Dataset Creation **4**
- 4.1 Input distributions . . . 4
- 4.2 z-pair and dz sampling . . . 4
- 4.3 Important V1 design correction . . . 4

5 Dataset Schema and Storage **4**
- 5.1 Per-sample tensors . . . 4
- 5.2 Batch merge protocol . . . 4

6 Large-scale Generation Procedure (50M Production) **5**
- 6.1 HTCondor submission for data generation . . . 5
- 6.2 Worker script behavior . . . 5

7 Current Model Architecture Protocol in Use **5**
- 7.1 Active model family . . . 5
- 7.2 MLP definition . . . 5
- 7.3 Normalization strategy . . . 6

8 Current Training Protocol in Use **6**
- 8.1 Loss and objective . . . 6
- 8.2 Data split and loaders . . . 6
- 8.3 Optimizer, scheduler, and stabilization . . . 6
- 8.4 Stopping and checkpointing . . . 6
- 8.5 Metrics tracked . . . 7

9 Current HTCondor Training Sweep Configuration **7**
9.1 Submit file settings . . . 7
9.2 Worker command template . . . 7
9.3 Sweep grid (17 architectures) . . . 7

10 Experiment Tracking and Artifacts **8**
10.1 MLflow . . . 8
10.2 On-disk outputs per run . . . 8

11 Reproducibility Notes and Operational Caveats **8**
11.1 Randomness control . . . 8
11.2 Known operational decisions reflected in current protocol . . . 8

12 Minimal Reproduction Commands **9**
12.1 Generate one local sample dataset . . . 9
12.2 Merge batch datasets . . . 9
12.3 Run one training job locally (MLP protocol) . . . 9

13 Protocol Summary **9**

1 Scope and Purpose

This document specifies, in implementation-level detail, how the current V1 pipeline creates data and trains models. It is designed to be exhaustive and reproducible from source scripts.

1.1 What is covered

- Physics and numerical equations used to generate ground truth.
- Field map source, interpolation method, and sampling domains.
- Exact dataset schema, dimensionality, and normalization context.
- HTCondor batch production strategy for the 50M-track dataset.
- Current model architecture and training protocol in active use.
- Current sweep configuration (hyperparameters and resource requests).

1.2 What is not currently active in V1 training

The codebase contains PINN and RK-PINN definitions in the architecture module, but the active V1 protocol currently trains pure MLP models with supervised MSE only. No physics residual loss is used in the current submitted jobs.

2 Code and File Inventory (Source of Truth)

The protocol is derived from the following files:

- `V1/data_generation/generate_data.py`
- `V1/data_generation/condor/datagen.sub`
- `V1/data_generation/condor/run_datagen.sh`
- `V1/data_generation/merge_batches.py`
- `V1/utils/magnetic_field.py`
- `V1/utils/rk4_propagator.py`
- `V1/models/architectures.py`
- `V1/models/train.py`
- `V1/models/condor/train_sweep.sub`
- `V1/models/condor/run_train.sh`

3 Physics Model Used for Data Generation

3.1 State Definition

The propagated state is

$$\mathbf{s}(z) = [x, y, t_x, t_y, q/p]^\top, \tag{1}$$

where $t_x = dx/dz$, $t_y = dy/dz$, and q/p is expressed in MeV^{-1}.

3.2 Lorentz-force equations in z-parameterization

The RK4 integrator implements:

$$\frac{dx}{dz} = t_x, \tag{2}$$

$$\frac{dy}{dz} = t_y, \tag{3}$$

$$\frac{dt_x}{dz} = \kappa N \left(t_x t_y B_x - (1 + t_x^2) B_y + t_y B_z \right), \tag{4}$$

$$\frac{dt_y}{dz} = \kappa N \left((1 + t_y^2) B_x - t_x t_y B_y - t_x B_z \right), \tag{5}$$

$$\frac{d(q/p)}{dz} = 0, \tag{6}$$

with

$$\kappa = c\,(q/p), \qquad c = 2.99792458 \times 10^{-4}, \qquad N = \sqrt{1 + t_x^2 + t_y^2}. \tag{7}$$

3.3 Magnetic field source and interpolation

- Field map file: `field_maps/twodip.rtf`.
- Grid dimensions: $81 \times 81 \times 146$.
- Grid spacing: 100 mm in each axis.
- Coordinate bounds: $x, y \in [-4000, 4000]$ mm, $z \in [-500, 14000]$ mm.
- Interpolation: trilinear (`scipy.interpolate.RegularGridInterpolator`).
- Outside-grid behavior: fill value 0.0 for each field component.
- Polarity in production: -1 (MagDown), applied to B_x and B_y sign in interpolated model.

3.4 Integrator settings

- Integrator: fixed-step classical RK4.
- Step size: 5 mm in production generation.
- Propagation direction: forward only in the current production setup ($z_{\text{end}} > z_{\text{start}}$).

3.5 Track-level validity checks

A generated track is marked invalid and discarded if any of the following occurs:

- Any non-finite state value during/after propagation.
- $|x| > 5000$ mm or $|y| > 5000$ mm during integration.
- $|t_x| > 2.0$ or $|t_y| > 2.0$ during integration.

Invalid tracks are returned as zero arrays and removed in post-processing by filtering on momentum $P > 0$.

4 Sampling Protocol for Dataset Creation

4.1 Input distributions

For each track, sampling is:

- $p \sim \log\mathcal{U}(1, 100)$ GeV.
- $q \in \{-1, +1\}$, equiprobable; $q/p = q/(1000\,p)$ in MeV^{-1}.
- $x \sim \mathcal{U}(-300, 300)$ mm.
- $y \sim \mathcal{U}(-250, 250)$ mm.
- $t_x \sim \mathcal{U}(-0.3, 0.3)$.
- $t_y \sim \mathcal{U}(-0.25, 0.25)$.

4.2 z-pair and dz sampling

- Base bounds: $z_{\min} = 0$, $z_{\max} = 14000$ mm.
- Draw $z_{\text{start}} \sim \mathcal{U}(z_{\min}, z_{\max} - dz_{\min})$.
- Draw $dz \sim \mathcal{U}(dz_{\min}, dz_{\max})$ with $dz_{\min} = 100$ mm, $dz_{\max} = 10000$ mm.
- Set $z_{\text{end}} = z_{\text{start}} + dz$, then clip to $[z_{\min} + dz_{\min}, z_{\max}]$.
- Recompute $dz = z_{\text{end}} - z_{\text{start}}$ and retain only samples with $dz \geq dz_{\min}$.

4.3 Important V1 design correction

This V1 pipeline explicitly uses **variable** dz per track. This avoids the old fixed-dz behavior that caused near-zero standard deviation in the dz input channel and poor normalization behavior when evaluating at other step sizes.

5 Dataset Schema and Storage

5.1 Per-sample tensors

Saved arrays (NumPy compressed `.npz`):

- $X \in \mathbb{R}^{N\times 6}$ (input):
$$X_i = [x, y, t_x, t_y, q/p, dz].$$
- $Y \in \mathbb{R}^{N\times 4}$ (target):
$$Y_i = [x_{\text{out}}, y_{\text{out}}, t_{x,\text{out}}, t_{y,\text{out}}].$$
- $P \in \mathbb{R}^{N}$: sampled momentum in GeV.
- $Z \in \mathbb{R}^{N\times 2}$: $[z_{\text{start}}, z_{\text{end}}]$.

5.2 Batch merge protocol

Batch files `batch_*.npz` are merged by concatenation along axis 0 for each array, with optional NaN/Inf verification. Final merged artifact is saved via `np.savez_compressed`.

6 Large-scale Generation Procedure (50M Production)

6.1 HTCondor submission for data generation

`datagen.sub` configuration:

- Universe: `vanilla`
- Executable: `/bin/bash`
- Job category: `medium`
- Resources per job: 2 CPUs, 4 GB RAM, 500 MB disk
- Queue size: 2000 jobs
- Tracks per job: 25,000

Total requested tracks:

$$N_{\text{total}} = 2000 \times 25,000 = 50,000,000. \tag{8}$$

6.2 Worker script behavior

`run_datagen.sh` performs:

1. Activate conda env `TE`.
2. Compute batch seed:
$$\texttt{seed} = 42 + \texttt{batch_id} \times 7919.$$
3. Run `generate_data.py` with production physics ranges, 2 workers, step size 5 mm, polarity -1.

Inside each worker process, NumPy RNG is reseeded from OS entropy to avoid identical forked random streams.

7 Current Model Architecture Protocol in Use

7.1 Active model family

The active submitted training protocol uses **MLP only**. Although PINN and RK-PINN are implemented in `architectures.py`, they are not used in the current running sweep.

7.2 MLP definition

For hidden dimensions $(h_1, \ldots, h_L)$, network is:

$$\hat{\mathbf{y}} = \text{Denorm}\left(W_{L+1}\,\phi_L\left(\cdots\phi_1(W_1\,\text{Norm}(\mathbf{x}) + b_1)\cdots\right) + b_{L+1}\right), \tag{9}$$

where:

- Input dimension is 6, output dimension is 4.
- Activation is SiLU in current runs.
- Dropout is 0.0 in current runs.
- Linear weights initialized with Xavier uniform; biases initialized to zero.

7.3 Normalization strategy

- Mean/std are computed on the training split only.
- Inputs are z-score normalized before forward pass.
- Outputs are predicted in normalized space and denormalized back to physical units.

8 Current Training Protocol in Use

8.1 Loss and objective

Current objective is supervised MSE over full 4D output state:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{B}\sum_{i=1}^{B} \|\hat{\mathbf{y}}_i - \mathbf{y}_i\|_2^2, \quad \mathbf{y}_i = [x, y, t_x, t_y]. \tag{10}$$

No PDE, IC, or auxiliary physics loss terms are included in current V1 training runs.

8.2 Data split and loaders

- Split fractions: train 0.8, validation 0.1, test 0.1.
- Split method: random permutation, then contiguous slicing by fraction.
- DataLoader:
  - Train: shuffled, `drop_last=True`.
  - Val/Test: no shuffle, full batches.

8.3 Optimizer, scheduler, and stabilization

- Optimizer: AdamW.
- Learning rate: 10^{-3}.
- Weight decay: 10^{-4}.
- Scheduler: cosine with warmup (warmup epochs = 5 in code defaults).
- Gradient clipping: global norm clip at 1.0.

8.4 Stopping and checkpointing

- Early stopping with patience and min-delta.
- Best model saved by validation loss to `best_model.pt`.
- Periodic checkpoints every `save_every` epochs.
- Normalization JSON persisted with best model.

8.5 Metrics tracked

Validation and test track:

- Mean position error: $\sqrt{(\Delta x)^2 + (\Delta y)^2}$ (mm).
- Position std and 95th percentile (mm).
- Mean slope error: $\sqrt{(\Delta t_x)^2 + (\Delta t_y)^2}$.
- Component-wise absolute means for x, y, t_x, t_y.

9 Current HTCondor Training Sweep Configuration

9.1 Submit file settings

From `train_sweep.sub`:

- Universe: `vanilla`, executable: `/bin/bash`.
- Job category: `short`.
- Resources per training job: 1 GPU, 1 CPU, 8 GB RAM.
- Shared filesystem mode (no transfer).

9.2 Worker command template

From `run_train.sh`, each job runs:

```
python train.py --model mlp --name <exp_name> --hidden_dims <dims...> \
  --epochs 500 --patience 30 --min_delta 1e-7 \
  --batch_size 4096 --lr 1e-3 --weight_decay 1e-4 \
  --activation silu --num_workers 0 \
  --mlflow --mlflow-experiment V1_MLP_sweep \
  --max_samples 5000000
```

9.3 Sweep grid (17 architectures)

`Experiment name`	Hidden dims	Max samples
`mlp_2x64`	[64, 64]	5,000,000
`mlp_2x128`	[128, 128]	5,000,000
`mlp_2x256`	[256, 256]	5,000,000
`mlp_2x512`	[512, 512]	5,000,000
`mlp_2x1024`	[1024, 1024]	5,000,000
`mlp_3x64`	[64, 64, 64]	5,000,000
`mlp_3x128`	[128, 128, 128]	5,000,000
`mlp_3x256`	[256, 256, 256]	5,000,000
`mlp_3x512`	[512, 512, 512]	5,000,000
`mlp_128_64`	[128, 64]	5,000,000
`mlp_256_128`	[256, 128]	5,000,000
`mlp_256_256_128`	[256, 256, 128]	5,000,000
`mlp_512_256_128`	[512, 256, 128]	5,000,000
`mlp_512_512_256`	[512, 512, 256]	5,000,000

Experiment name	Hidden dims	Max samples
mlp_4x128	[128, 128, 128, 128]	5,000,000
mlp_4x256	[256, 256, 256, 256]	5,000,000
mlp_1024_512_256	[1024, 512, 256]	5,000,000

10 Experiment Tracking and Artifacts

10.1 MLflow

Current runs enable MLflow with experiment name `V1_MLP_sweep`. Logged entities include:

- Full scalar config parameters.
- Per-epoch metrics (train/val losses and validation errors).
- Final test metrics and best epoch/loss.
- Artifacts: best model, normalization, config, history, model config.

10.2 On-disk outputs per run

Each run produces an experiment directory under checkpoints containing at least:

- `best_model.pt`
- `normalization.json`
- `config.json`
- `history.json`
- `model_config.json`

11 Reproducibility Notes and Operational Caveats

11.1 Randomness control

- Dataset script has top-level seed argument (default 42).
- Batch jobs use deterministic seed offset per batch.
- Worker-level RNG reseeding from OS entropy prevents forked-stream duplication.

11.2 Known operational decisions reflected in current protocol

- Training jobs use `num_workers=0` to match 1-CPU GPU slot requests.
- Sweep argument format uses underscore-separated hidden dims in Condor queue rows, converted to space-separated list in worker script.
- Job category is `short` to satisfy cluster scheduling windows.

12 Minimal Reproduction Commands

12.1 Generate one local sample dataset

```
cd V1/data_generation
python generate_data.py --n-tracks 100000 --name train_100k --output-dir datasets
```

12.2 Merge batch datasets

```
cd V1/data_generation
python merge_batches.py --input "datasets/batch_*.npz" --output datasets/train_50M.npz
     --verify
```

12.3 Run one training job locally (MLP protocol)

```
cd V1/models
python train.py --model mlp --name mlp_test --hidden_dims 256 256 128 \
  --epochs 500 --patience 30 --min_delta 1e-7 \
  --batch_size 4096 --lr 1e-3 --weight_decay 1e-4 \
  --activation silu --num_workers 0 --mlflow \
  --mlflow-experiment V1_MLP_sweep --max_samples 5000000
```

13 Protocol Summary

The current V1 production protocol is:

1. Generate RK4 truth tracks using interpolated real field map `twodip.rtf` with variable dz.
2. Store samples in (X, Y, P, Z) schema with $X = [x, y, t_x, t_y, q/p, dz]$ and $Y = [x, y, t_x, t_y]$ at propagated endpoint.
3. Produce large datasets via Condor arrays and merge compressed NPZ batches.
4. Train MLP models only (for now) using supervised MSE on full 4D output, with AdamW, cosine schedule, early stopping, and MLflow tracking.
5. Execute a 17-architecture GPU sweep at 5M samples per run to identify best-performing MLP families before any full-50M retraining.